

Data Analytics

Professional Study Notes

Intermediate Level | SQL · Python · Visualization · ML · BI · CRISP-DM

Table of Contents

01 SQL & Databases

- Joins & subqueries
- Window functions
- CTEs
- Outlier detection
- Data quality SQL

02 Python for Analytics

- Pandas essentials
- NumPy
- EDA workflow
- Data cleaning
- Merging & reshaping

03 Data Visualization

- Matplotlib
- Seaborn
- Plotly
- Chart selection
- Storytelling with data

04 Machine Learning Basics

- Supervised vs unsupervised
- Regression
- Classification
- Clustering
- Model evaluation

05 Business Intelligence & Dashboards

- BI concepts
- KPIs & metrics
- Dashboard design
- Tools overview

06 CRISP-DM & Project Workflow

- All 6 phases
- Deliverables
- Iteration patterns
- Professional tips

CHAPTER 01

SQL & Databases

1.1 Core SQL Concepts

SQL (Structured Query Language) is the primary language for querying and manipulating relational databases. At an intermediate level you should be fluent in SELECT, filtering, aggregation, joins, subqueries, window functions, and CTEs.

Anatomy of a SELECT statement

```
SELECT column1, column2, AGG_FUNC(column3) AS alias
FROM schema.table
JOIN other_table ON table.id = other_table.fk
WHERE condition
GROUP BY column1, column2
HAVING AGG_FUNC(column3) > value
ORDER BY column1 DESC
LIMIT 100;
```

Note: HAVING filters AFTER aggregation; WHERE filters BEFORE. Never use WHERE on an aggregate.

1.2 Joins

Join Type	Returns	Use When
INNER JOIN	Only matching rows in both tables	You only want records with a match
LEFT JOIN	All left rows + matched right rows (NULLs)	Keep all records from left table
RIGHT JOIN	All right rows + matched left rows (NULLs)	Keep all records from right table
FULL OUTER	All rows from both, NULLs where no match	See all data from both sides
CROSS JOIN	Cartesian product of both tables	Generating combinations
SELF JOIN	Table joined to itself	Hierarchies, comparing rows

```
-- LEFT JOIN example: all customers, even those with no orders
SELECT c.CustomerID, c.Name, COUNT(o.OrderID) AS total_orders
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name;
```

1.3 Subqueries

A subquery is a SELECT statement nested inside another query. It can appear in SELECT, FROM, or WHERE clauses.

```
-- Correlated subquery: customers who spent above average
SELECT CustomerID, TotalSpend
FROM Sales
```

```
WHERE TotalSpend > (SELECT AVG(TotalSpend) FROM Sales);  
-- Subquery in FROM (derived table)  
SELECT dept, avg_sal FROM (  
  SELECT Department AS dept, AVG(Salary) AS avg_sal  
  FROM Employees  
  GROUP BY Department  
) dept_avg  
WHERE avg_sal > 60000;
```

1.4 Common Table Expressions (CTEs)

CTEs (WITH clauses) make complex queries readable by naming intermediate result sets. They are evaluated once and can be referenced multiple times.

```
WITH monthly_sales AS (  
  SELECT DATE_TRUNC('month', OrderDate) AS month,  
  SUM(Amount) AS revenue  
  FROM Orders  
  GROUP BY 1  
) ,  
ranked AS (  
  SELECT *, RANK() OVER (ORDER BY revenue DESC) AS rnk  
  FROM monthly_sales  
)  
SELECT * FROM ranked WHERE rnk <= 3;
```

1.5 Window Functions

Window functions perform calculations across a set of rows related to the current row — without collapsing rows like GROUP BY does.

Function	Purpose	Example
ROW_NUMBER())	Unique sequential integer per row	ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC)
RANK()	Rank with gaps for ties	RANK() OVER (ORDER BY score DESC)
DENSE_RANK()	Rank without gaps	DENSE_RANK() OVER (ORDER BY score DESC)
LAG(col,n)	Value n rows before current	LAG(revenue,1) OVER (ORDER BY month)
LEAD(col,n)	Value n rows after current	LEAD(revenue,1) OVER (ORDER BY month)
SUM() OVER()	Running / cumulative total	SUM(amount) OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)
AVG() OVER()	Moving average	AVG(amount) OVER (ORDER BY date ROWS 6 PRECEDING)

```
-- Month-over-month revenue change  
SELECT month, revenue,  
LAG(revenue) OVER (ORDER BY month) AS prev_month,
```

```
revenue - LAG(revenue) OVER (ORDER BY month) AS change
FROM monthly_sales;
```

1.6 Outlier Detection in SQL

```
-- Method 1: Simple threshold (> 3x average)
SELECT * FROM Orders
WHERE Amount > 3 * (SELECT AVG(Amount) FROM Orders);

-- Method 2: Z-score (flag if |z| > 3)
SELECT *, (Amount - AVG(Amount) OVER()) /
NULLIF(STDDEV(Amount) OVER(), 0) AS z_score
FROM Orders;

-- Method 3: IQR method
WITH stats AS (
  SELECT PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY Amount) AS q1,
  PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY Amount) AS q3
  FROM Orders
)
SELECT o.* FROM Orders o, stats
WHERE o.Amount < stats.q1 - 1.5*(stats.q3-stats.q1)
OR o.Amount > stats.q3 + 1.5*(stats.q3-stats.q1);
```

1.7 Data Quality SQL Patterns

```
-- Count NULLs per column
SELECT COUNT(*) - COUNT(col1) AS null_col1,
COUNT(*) - COUNT(col2) AS null_col2
FROM my_table;

-- Find duplicates
SELECT email, COUNT(*) AS cnt
FROM Customers
GROUP BY email HAVING COUNT(*) > 1;

-- Delete duplicates, keep lowest ID
DELETE FROM Customers
WHERE CustomerID NOT IN (
  SELECT MIN(CustomerID) FROM Customers GROUP BY Email);
```

CHAPTER 02

Python for Analytics

2.1 Pandas Essentials

Pandas is the backbone of Python-based data analysis. Master these operations and you can handle 90% of real-world data tasks.

Loading & inspecting data

```
import pandas as pd

df = pd.read_csv('data.csv')
df.head() # first 5 rows
df.info() # dtypes, non-null counts
df.describe() # summary statistics
df.shape # (rows, cols)
df.columns.tolist() # column names
df.dtypes # data types
```

Selecting & filtering

```
df['col'] # Series
df[['col1', 'col2']] # DataFrame
df.loc[0:5, 'col1':'col3'] # label-based slice
df.iloc[0:5, 0:3] # position-based slice
df[df['Age'] > 30] # boolean filter
df.query('Age > 30 and City=="London"') # query syntax
```

Aggregation & groupby

```
df.groupby('Category')['Sales'].sum()
df.groupby('Category').agg(
    total=('Sales', 'sum'),
    avg=('Sales', 'mean'),
    count=('Sales', 'count')
)
df.pivot_table(values='Sales', index='Region',
    columns='Category', aggfunc='sum')
```

2.2 Data Cleaning in Python

```
# Missing values
df.isnull().sum() # count NULLs
df.dropna(subset=['email']) # drop rows with null email
df['age'].fillna(df['age'].median(), inplace=True)

# Duplicates
df.duplicated().sum() # count duplicates
df.drop_duplicates(subset=['email'], keep='first', inplace=True)
```

```
# Data types
df['date'] = pd.to_datetime(df['date'])
df['price'] = pd.to_numeric(df['price'], errors='coerce')

# String cleaning
df['email'] = df['email'].str.strip().str.lower()
df['city'] = df['city'].str.replace(' ', ' ').str.title()
```

2.3 EDA Workflow

A structured EDA process prevents missed insights. Follow this sequence on every new dataset:

- Step 1: Shape check: `df.shape`, `df.info()`
- Step 2: Missing values: `df.isnull().sum() / df.isnull().mean()*100`
- Step 3: Distributions: `df.describe()`, histograms for numeric cols
- Step 4: Categorical cardinality: `df['col'].value_counts()`
- Step 5: Outliers: boxplots, z-score checks
- Step 6: Correlations: `df.corr()`, heatmap
- Step 7: Target variable analysis: distribution of what you're predicting

2.4 Merging & Reshaping

```
# Merge (like SQL JOIN)
merged = pd.merge(df1, df2, on='CustomerID', how='left')

# Concat (stack rows or columns)
combined = pd.concat([df_2023, df_2024], ignore_index=True)

# Melt: wide to long
df_long = df.melt(id_vars=['ID'], value_vars=['Q1', 'Q2', 'Q3'],
var_name='Quarter', value_name='Revenue')

# Pivot: long to wide
df_wide = df_long.pivot(index='ID', columns='Quarter', values='Revenue')
```

2.5 NumPy for Analytics

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
np.mean(arr), np.median(arr), np.std(arr)
np.percentile(arr, [25, 50, 75]) # quartiles

# Vectorised operations (much faster than loops)
prices = df['Price'].values
z_scores = (prices - np.mean(prices)) / np.std(prices)
outliers = df[np.abs(z_scores) > 3]
```

CHAPTER 03

Data Visualization

3.1 Chart Selection Guide

Goal	Best Chart	When to avoid
Compare categories	Bar / column chart	Too many categories (>12)
Show trend over time	Line chart	Non-continuous time data
Show distribution	Histogram / box plot	Small samples (<20 values)
Show correlation	Scatter plot	Too many points (use hexbin)
Show composition	Stacked bar / pie	Pie if >5 slices
Show geographic data	Choropleth map	When geography is irrelevant
Show relationships	Heatmap / correlation	Too many variables
Compare distributions	Box plot / violin plot	When exact values matter

3.2 Matplotlib

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Bar chart
axes[0].bar(df['Category'], df['Sales'], color='steelblue')
axes[0].set_title('Sales by Category')
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Sales')

# Line chart with markers
axes[1].plot(df['Month'], df['Revenue'], marker='o', linewidth=2)
axes[1].set_title('Monthly Revenue')

plt.tight_layout()
plt.savefig('charts.png', dpi=150, bbox_inches='tight')
plt.show()
```

3.3 Seaborn

Seaborn wraps Matplotlib with better defaults and statistical chart types. Ideal for EDA.

```
import seaborn as sns

# Distribution
sns.histplot(df['Age'], kde=True, bins=30)
```



```
# Boxplot by group
sns.boxplot(x='Department', y='Salary', data=df, palette='Set2')

# Correlation heatmap
corr = df.select_dtypes(include='number').corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')

# Pairplot (EDA overview)
sns.pairplot(df, hue='Category', diag_kind='kde')
```

3.4 Plotly (Interactive Charts)

```
import plotly.express as px

# Interactive bar
fig = px.bar(df, x='Category', y='Sales', color='Region',
             title='Sales by Category and Region')
fig.show()

# Interactive scatter
fig = px.scatter(df, x='Age', y='Income', color='Gender',
                 size='Purchases', hover_data=['CustomerID'])
fig.show()

# Time series
fig = px.line(df, x='Date', y='Revenue', color='Product')
fig.update_layout(hovermode='x unified')
```

3.5 Storytelling with Data

Key principles every analyst should apply:

- Start with the insight, not the data — lead with the conclusion, support with the chart
- Reduce cognitive load — remove gridlines, borders, and axis clutter that don't add meaning
- Use colour intentionally — highlight the one thing that matters, grey out the rest
- Label directly — annotations on the chart beat a legend whenever possible
- Context beats raw numbers — always show a comparison (vs last year, vs target, vs average)
- Know your audience — executives want the takeaway; analysts want the detail

CHAPTER 04

Machine Learning Basics

4.1 ML Taxonomy

Type	Definition	Examples
Supervised	Learn from labelled data (input + known output)	Regression, Classification
Unsupervised	Find patterns in unlabelled data	Clustering, Dimensionality reduction
Semi-supervised	Mix of labelled and unlabelled data	Self-training, label propagation
Reinforcement	Agent learns via rewards and penalties	Game AI, recommendation engines

4.2 Regression

Regression predicts a continuous numeric output. Core algorithms and their use cases:

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f'MAE: {mean_absolute_error(y_test, y_pred):.2f}')
print(f'RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}')
print(f'R2: {r2_score(y_test, y_pred):.3f}')
```

4.3 Classification

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report,
                             confusion_matrix, roc_auc_score)

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test)[:,1]

print(classification_report(y_test, y_pred))
print('AUC-ROC:', roc_auc_score(y_test, y_proba))
```

Classification metrics cheat sheet

Metric	Formula	Best When
Accuracy	Correct / Total	Balanced classes
Precision	$TP / (TP + FP)$	Cost of false positives is high
Recall	$TP / (TP + FN)$	Cost of false negatives is high (e.g. cancer)
F1 Score	$2 * P * R / (P + R)$	Imbalanced classes
AUC-ROC	Area under ROC curve	Comparing models, imbalanced data

4.4 Clustering

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Always scale before clustering
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Find optimal k with elbow method
inertias = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

# Fit final model
km = KMeans(n_clusters=4, random_state=42, n_init=10)
df['cluster'] = km.fit_predict(X_scaled)
print('Silhouette:', silhouette_score(X_scaled, df['cluster']))
```

4.5 Model Evaluation Best Practices

- Always hold out a test set and never touch it until final evaluation
- Use cross-validation (cv=5 or cv=10) for more reliable performance estimates
- Scale features before distance-based algorithms (KNN, SVM, clustering)
- Check for data leakage — future information must never enter training features
- Report multiple metrics — accuracy alone is meaningless on imbalanced data
- Examine feature importance to validate that model logic makes business sense
- Baseline always — compare your model to the simplest possible approach (e.g. predict the mean)

CHAPTER 05

Business Intelligence & Dashboards

5.1 BI Core Concepts

Term	Definition
KPI (Key Performance Indicator)	Quantifiable measure tied directly to a strategic goal
Dimension	Categorical attribute used to slice data (Region, Product, Date)
Measure / Metric	Numeric value that is aggregated (Revenue, Units Sold, NPS)
Granularity	The level of detail in a row (transaction, daily, monthly)
Star schema	Fact table at centre with surrounding dimension tables
Snowflake schema	Star schema where dimension tables are further normalised
OLTP	Optimised for transactional writes (e.g. a sales system)
OLAP	Optimised for analytical reads (e.g. a data warehouse)
Data mart	Subject-specific subset of a data warehouse
ETL / ELT	Extract-Transform-Load / Extract-Load-Transform pipelines

5.2 KPI Design

A well-designed KPI passes the SMART test:

- **Specific** — Measures exactly one thing — 'Monthly Recurring Revenue' not 'revenue'
- **Measurable** — Has a reliable, automated data source
- **Achievable** — Represents a realistic target
- **Relevant** — Directly tied to a business decision
- **Time-bound** — Has a defined reporting cadence and time window

Common analytics KPIs by domain:

Domain	Key KPIs
E-commerce	Conversion rate, AOV, CAC, CLV, churn rate, cart abandonment
Marketing	CTR, CPC, ROAS, MQL, SQL, lead-to-close rate
Finance	Gross margin, EBITDA, burn rate, ARR, MRR
Operations	Throughput, defect rate, cycle time, OEE, SLA compliance
Product	DAU/MAU, retention rate, NPS, feature adoption, time-to-value

5.3 Dashboard Design Principles

- Answer one question per dashboard — 'How is our sales performance this month?'
- Put the most important KPI top-left — users read F-pattern
- Use consistent colours across all charts on the same dashboard
- Always show context: actuals vs target, vs prior period, vs forecast
- Limit to 5-7 metrics on a single view — more causes decision fatigue
- Make filters visible and obvious — user should always know what time range they're looking at
- Design for the worst screen — a dashboard that only works on a 4K monitor will fail

5.4 BI Tools Overview

Tool	Strengths	Best For
Power BI	Deep Microsoft integration, DAX, affordable	Enterprise, Office 365 shops
Tableau	Best-in-class visualisation, drag-and-drop	Visual analytics, large orgs
Looker	Git-based modelling (LookML), semantic layer	Data-mature engineering teams
Metabase	Open source, easy self-serve, SQL-first	Startups, technical teams
Grafana	Time-series, infrastructure, real-time	Engineering / DevOps metrics
Python Dash	Fully custom, programmable, free	Bespoke analytical apps

CHAPTER 06

CRISP-DM & Project Workflow

6.1 The Six Phases

01 Business Understanding

Define the problem in business terms. Set measurable success criteria. Confirm data can answer the question. Output: project charter, success criteria.

- What business decision will this analysis support?
- What does 'success' look like — and how will we measure it?
- What is the minimum viable dataset needed?
- What are the constraints (budget, timeline, privacy)?

02 Data Understanding

Collect, describe, explore, and quality-check the data. Output: data understanding report.

- `df.info()`, `df.describe()`, `df.isnull().sum()`
- Distribution plots for all numeric columns
- Cardinality check on all categorical columns
- Outlier flagging with IQR / z-score
- Correlation matrix

03 Data Preparation

Clean, transform, engineer features, and build the analytical dataset. ~60-80% of project time. Output: model-ready dataset.

- Handle missing values (impute or drop)
- Remove / flag outliers
- Encode categoricals (one-hot, ordinal, target encoding)
- Scale numerics (StandardScaler, MinMaxScaler)
- Engineer features (ratios, lags, rolling averages)
- Train / validation / test split

04 Modeling

Select algorithms, design test strategy, build and tune models. Output: candidate models + assessment.

- Start with a simple baseline (mean, logistic regression)
- Try 2-3 algorithm families
- Use cross-validation for honest estimates
- Tune with GridSearchCV or RandomizedSearchCV
- Record all experiments (MLflow, spreadsheet, or notebook)

05 Evaluation

Assess model against business success criteria — not just accuracy. Output: go/no-go decision.

- Does the model answer the original business question?
- Are the predictions interpretable to stakeholders?
- What is the expected business impact?
- Are there ethical or fairness concerns?
- Is the model stable across time periods and segments?

06 Deployment

Integrate into production. Monitor, maintain, and retrain. Output: live model or report in production.

- REST API (FastAPI / Flask)
- Batch scoring pipeline (scheduled SQL + model)
- Dashboard integration
- Monitoring: data drift, model drift, alerting
- Retraining schedule and ownership

6.2 Common Iteration Patterns

Trigger	Loop Back To	Why
EDA reveals data can't answer question	Business Understanding	Redefine scope or find better data
Modeling exposes data quality issues	Data Preparation	Fix leakage, fix errors, re-engineer
Model technically good but business rejects	Business Understanding	Success criteria misaligned
Production data drifts from training data	Data Understanding	Retrain with fresh data

6.3 Professional Tips

- Version control your notebooks and data pipelines with Git from day one
- Document every data transformation — if you can't explain it, you can't defend it
- Build reproducible pipelines: same code + same data must always produce same result
- Communicate uncertainty — never present a model's output as ground truth
- Involve business stakeholders early — surprises at deployment are expensive
- Separate exploration notebooks from production code
- Log all model experiments — you will need to compare runs six months later
- Think about the end user of every chart or model output you produce